Experiment No.5
Study and Implementation of Lexical Analyzer.
Name: Harsh Nalawade
Roll no.38
Date of Performance:14/1/26
Date of Submission:21/1/26

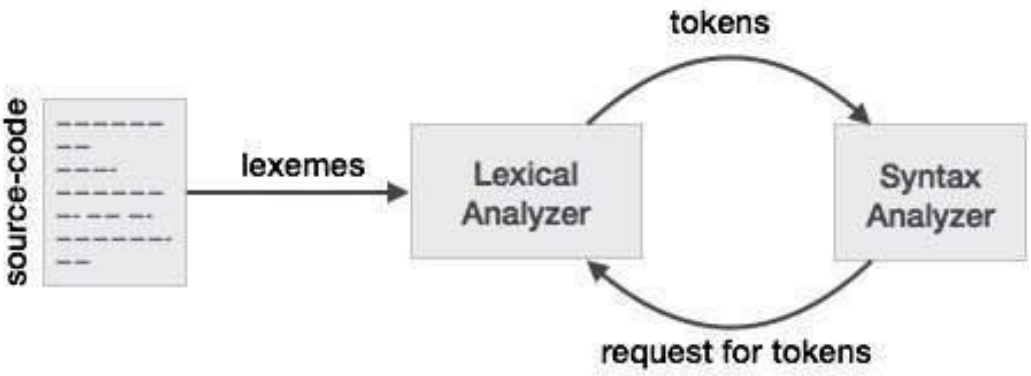
Aim: Study and Implementation of Lexical Analyzer.

Objective: To study and implement lexical analyzer program for identifying tokens from given input source program.

Theory:

Lexical analysis is the first phase of a compiler. It takes the modified source code from language preprocessors that are written in the form of sentences. The lexical analyzer breaks these syntaxes into a series of tokens, by removing any whitespace or comments in the source code.

If the lexical analyzer finds a token invalid, it generates an error. The lexical analyzer works closely with the syntax analyzer. It reads character streams from the source code, checks for legal tokens, and passes the data to the syntax analyzer when it demands.



Lexical analysis : process of taking an input string of characters (such as the source code of a computer program) and producing a sequence of symbols called lexical tokens, or just tokens, which may be handled more easily by a parser. The pattern matches each string in the set.

Tokens:

Lexemes are said to be a sequence of characters (alphanumeric) in a token. There are some predefined rules for every lexeme to be identified as a valid token. These rules are defined by grammar rules, by means of a pattern. A pattern explains what a token can be, and these patterns are defined by means of regular expressions.

In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.

For example, in C language, the variable declaration line
int value = 100;
contains the tokens:
int (keyword), value (identifier), = (operator), 100 (constant) and; (symbol).

Specifications of Tokens:

Let us understand how the language theory undertakes the following terms:

Alphabets

Any finite set of symbols {0,1} is a set of binary alphabets, {0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F} is a set of Hexadecimal alphabets, {a-z, A-Z} is a set of English language alphabets.

Strings

Any finite sequence of alphabets is called a string. Length of the string is the total number of occurrence of alphabets, e.g., the length of the string tutorialspoint is 14 and is denoted by |tutorialspoint| = 14. A string having no alphabets, i.e. a string of zero length is known as an empty string and is denoted by ε (epsilon).

Special Symbols

A typical high-level language contains the following symbols:-

Arithmetic Symbols	Addition(+), Subtraction(-), Modulo(%), Multiplication(*), Division(/)
Punctuation	Comma(,), Semicolon(;), Dot(.), Arrow(->)
Assignment	=
Special Assignment	+=, /=, *=, -=
Comparison	==, !=, <, <=, >, >=
Preprocessor	#
Location Specifier	&
Logical	&, &&, |, ||, !
Shift Operator	>>, >>>, <<, <<<

Language

A language is considered as a finite set of strings over some finite set of alphabets. Computer languages are considered as finite sets, and mathematically set operations can be performed on them. Finite languages can be described by means of regular expressions.

Code:

```
#include <stdio.h>

#include <string.h>

#include <stdlib.h>

#include <ctype.h>


#define MAX_RULES 20

#define MAX_LEN 100


typedef struct {

    char lhs;

    char rhs[MAX_LEN];

} Rule;


Rule grammar[MAX_RULES];

int rule_count = 0;

char input[MAX_LEN];

int pos = 0;


// Function to add rules

void addRule(char *ruleStr) {

    char *arrow = strstr(ruleStr, "->");

    if (arrow != NULL) {
```

```
        grammar[rule_count].lhs = ruleStr[0];
        strcpy(grammar[rule_count].rhs, arrow + 2);
        rule_count++;
    }
}

// Function to print the tree branches visually
void printTreeLine(int level, int isLast) {
    for (int i = 0; i < level - 1; i++) {
        printf("│   ");
    }
    if (level > 0) {
        if (isLast) printf("└── ");
        else printf("├── ");
    }
}

// Core Recursive Descent Logic
int parse(char symbol, int level, int isLast) {
    int start_pos = pos;

    // 1. If it's a Terminal
    if (!isupper(symbol)) {
        if (input[pos] == symbol) {
            printTreeLine(level, isLast);
            printf("%c (match)\n", symbol);
```

```
        pos++;
        return 1;
    }
    return 0; // Failed match
}

// 2. If it's a Non-Terminal
// We need to find a rule that works before we commit to printing the tree branch
for (int i = 0; i < rule_count; i++) {
    if (grammar[i].lhs == symbol) {
        int current_pos = pos;
        int possible = 1;
        char *rhs = grammar[i].rhs;

        // Tentative check: see if this rule can actually be completed
        // (Standard RDP with backtracking)
        for (int j = 0; rhs[j] != '\0'; j++) {
            // This is a simplified check for the tree visualization
            // In a full parser, we'd use a temporary buffer to store the tree nodes
        }

        // Since we want a "Correct" tree, we print the node and then its children
        printTreeLine(level, isLast);
        printf("%c\n", symbol);

        int match_all = 1;
```

```
            int rhs_len = strlen(rhs);
            for (int j = 0; j < rhs_len; j++) {
                if (!parse(rhs[j], level + 1, j == rhs_len - 1)) {
                    match_all = 0;
                    break;
                }
            }

            if (match_all) return 1;

            // If rule failed, backtrack
            pos = current_pos;
            // Note: In a production parser, you'd "erase" the printed lines
            // or use a Tree Data Structure to store nodes before printing.
        }
    }
    return 0;
}

int main() {
    int n;
    char buffer[MAX_LEN];

    printf("--- Recursive Descent Parser with Visual Tree ---\n");
    printf("Enter number of rules: ");
    scanf("%d", &n);
```

```
    printf("Enter rules (e.g., S->aAb):\n");
    for (int i = 0; i < n; i++) {
        scanf("%s", buffer);
        addRule(buffer);
    }

    printf("\nEnter string to parse: ");
    scanf("%s", input);

    printf("\nGenerated Parse Tree:\n");
    if (parse(grammar[0].lhs, 0, 1) && input[pos] == '\0') {
        printf("\nResult: SUCCESS. The string follows the grammar.\n");
    } else {
        printf("\nResult: REJECTED.\n");
    }

    return 0;
}
```

Output:

```
--- Recursive Descent Parser with Visual Tree ---
Enter number of rules: 3
Enter rules (e.g., S->aAb):
S->aA
A->bB
B->c

Enter string to parse: abc

Generated Parse Tree:
S
├── a (match)
└── A
|   ├── b (match)
|   └── B
|   |   └── c (match)

Result: SUCCESS. The string follows the grammar.
```

Conclusion:

The lexical analyzer reads the source program and converts it into a sequence of tokens by removing white spaces and grouping characters into meaningful lexemes. It identifies keywords, identifiers, constants, operators, special symbols, and strings based on predefined rules. The program successfully classifies each token into its correct lexical category and performs the first phase of the compilation process.